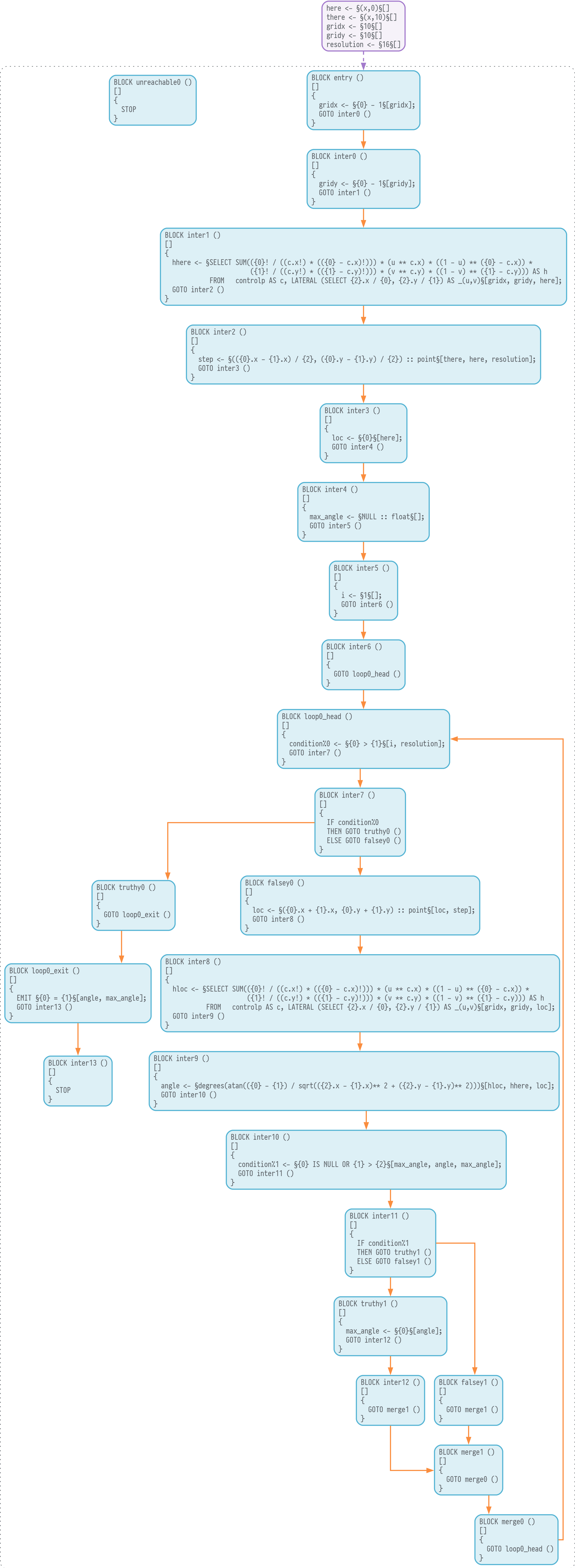
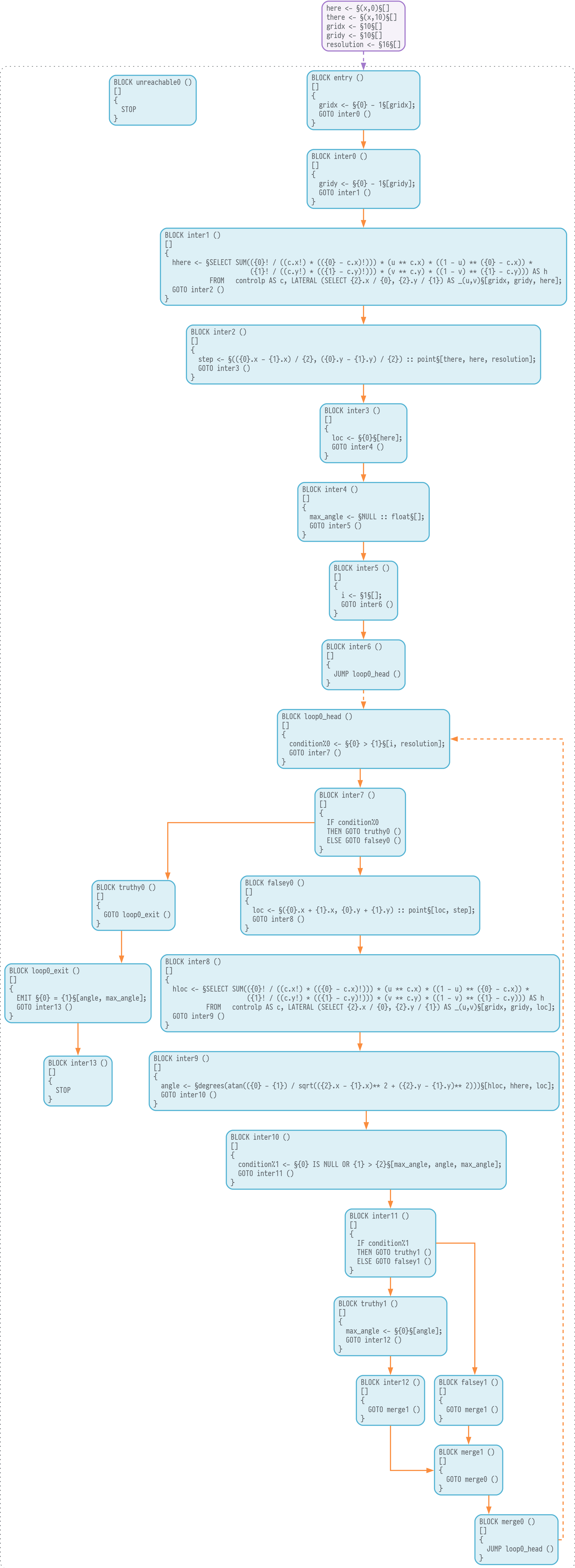







```
here <- $(x,0)$[]
there <- $(x,10)$[]
gridx <- $10$[]
gridy <- $10$[]
resolution <- $16$[]
```

```
BLOCK unreachable0 ()
[]
{
  STOP
}
```

```
BLOCK entry ()
[]
{
  gridx <- ${0} - 1$[gridx];
  gridy <- ${0} - 1$[gridy];
  step <- $(( ${0}.x - {1}.x ) / {2}, ( ${0}.y - {1}.y ) / {2}) :: point$[there, here, resolution];
  loc <- ${0}$[here];
  max_angle <- $NULL :: float$[];
  i <- $1$[];
  GOTO inter0 ()
}
```

```
BLOCK inter0 ()
[]
{
  hhere <- $SELECT SUM(( ${0}! / ((c.x!) * (( ${0} - c.x)!))) * (u ** c.x) * ((1 - u) ** ( ${0} - c.x)) *
    ( ${1}! / ((c.y!) * (( ${1} - c.y)!))) * (v ** c.y) * ((1 - v) ** ( ${1} - c.y))) AS h
    FROM   controlp AS c, LATERAL (SELECT {2}.x / {0}, {2}.y / {1}) AS _(u,v)$[gridx, gridy, here];
  GOTO inter1 ()
}
```

```
BLOCK inter1 ()
[]
{
  GOTO inter2 ()
}
```

```
BLOCK inter2 ()
[]
{
  GOTO inter3 ()
}
```

```
BLOCK inter3 ()
[]
{
  GOTO inter4 ()
}
```

```
BLOCK inter4 ()
[]
{
  GOTO inter5 ()
}
```

```
BLOCK inter5 ()
[]
{
  GOTO inter6 ()
}
```

```
BLOCK inter6 ()
[]
{
  JUMP loop0_head ()
}
```

```
BLOCK loop0_head ()
[]
{
  condition0 <- ${0} > {1}$[i, resolution];
  GOTO inter7 ()
}
```

```
BLOCK inter7 ()
[]
{
  IF condition0
  THEN GOTO truthy0 ()
  ELSE GOTO falsey0 ()
}
```

```
BLOCK truthy0 ()
[]
{
  EMIT ${0} = {1}$[angle, max_angle];
  GOTO loop0_exit ()
}
```

```
BLOCK falsey0 ()
[]
{
  loc <- $({0}.x + {1}.x, {0}.y + {1}.y) :: point$[loc, step];
  GOTO inter8 ()
}
```

```
BLOCK loop0_exit ()
[]
{
  GOTO inter13 ()
}
```

```
BLOCK inter8 ()
[]
{
  hloc <- $SELECT SUM(( ${0}! / ((c.x!) * (( ${0} - c.x)!))) * (u ** c.x) * ((1 - u) ** ( ${0} - c.x)) *
    ( ${1}! / ((c.y!) * (( ${1} - c.y)!))) * (v ** c.y) * ((1 - v) ** ( ${1} - c.y))) AS h
    FROM   controlp AS c, LATERAL (SELECT {2}.x / {0}, {2}.y / {1}) AS _(u,v)$[gridx, gridy, loc];
  GOTO inter9 ()
}
```

```
BLOCK inter13 ()
[]
{
  STOP
}
```

```
BLOCK inter9 ()
[]
{
  angle <- $degrees(atan(( ${0} - {1}) / sqrt(({2}.x - {1}.x)** 2 + ({2}.y - {1}.y)** 2))$[hloc, hhere, loc];
  GOTO inter10 ()
}
```

```
BLOCK inter10 ()
[]
{
  condition1 <- ${0} IS NULL OR {1} > {2}$[max_angle, angle, max_angle];
  GOTO inter11 ()
}
```

```
BLOCK inter11 ()
[]
{
  IF condition1
  THEN GOTO truthy1 ()
  ELSE GOTO falsey1 ()
}
```

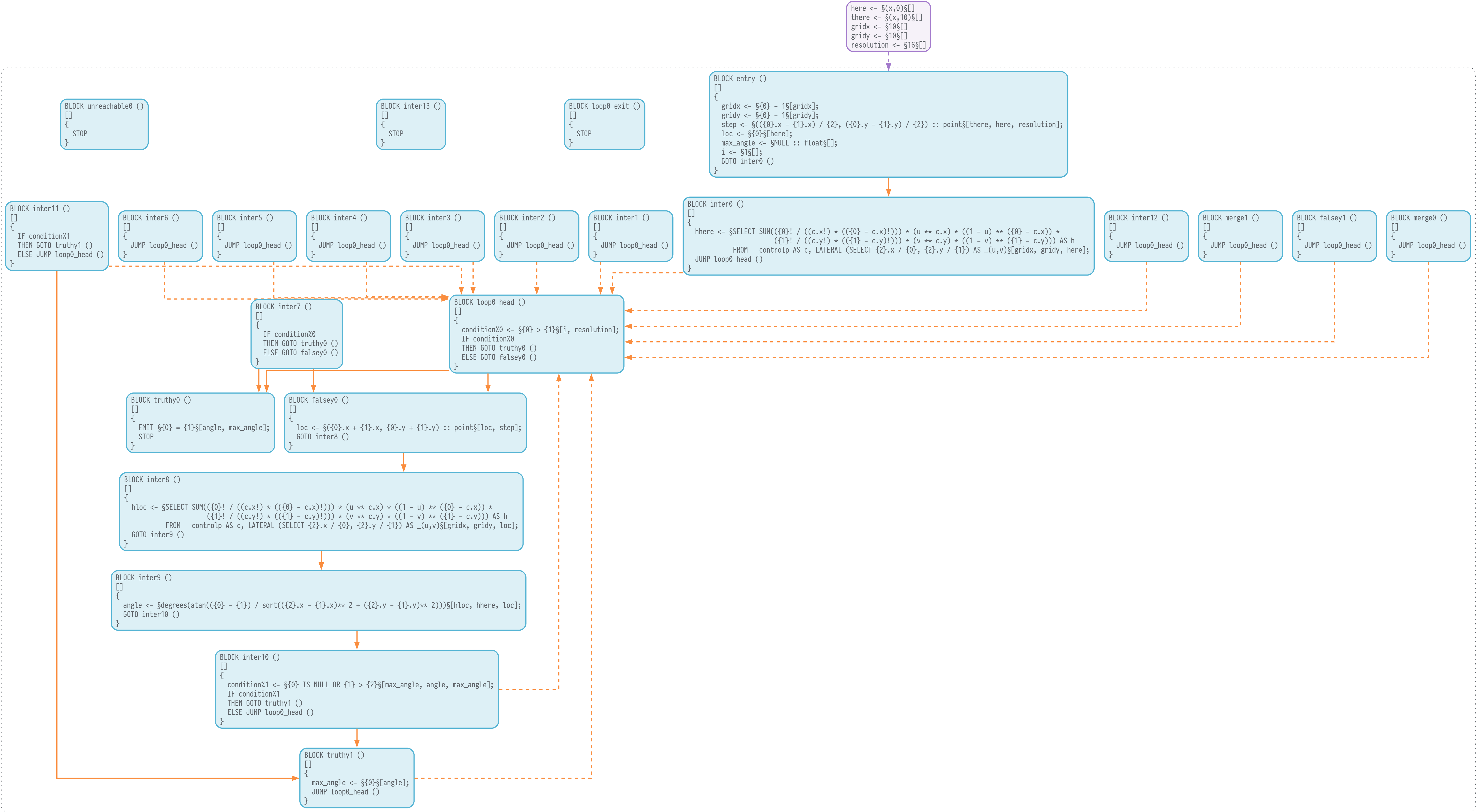
```
BLOCK truthy1 ()
[]
{
  max_angle <- ${0}$[angle];
  GOTO inter12 ()
}
```

```
BLOCK inter12 ()
[]
{
  GOTO merge1 ()
}
```

```
BLOCK falsey1 ()
[]
{
  GOTO merge1 ()
}
```

```
BLOCK merge1 ()
[]
{
  GOTO merge0 ()
}
```

```
BLOCK merge0 ()
[]
{
  JUMP loop0_head ()
}
```

```
here <- $(x,0)$[]
there <- $(x,10)$[]
gridx <- $10$[]
gridy <- $10$[]
resolution <- $16$[]
```

```
BLOCK entry ()
[]
{
  gridx <- ${0} - 1$[gridx];
  gridy <- ${0} - 1$[gridy];
  step <- $(( ${0}.x - {1}.x ) / {2}, ( ${0}.y - {1}.y ) / {2}) :: point$[there, here, resolution];
  loc <- ${0}$[here];
  max_angle <- $NULL :: float$[];
  i <- $1$[];
  GOTO inter0 ()
}
```

```
BLOCK inter0 ()
[]
{
  hhere <- $SELECT SUM(( ${0}! / ((c.x!) * (( ${0} - c.x)!))) * (u ** c.x) * ((1 - u) ** ( ${0} - c.x)) *
    ( ${1}! / ((c.y!) * (( ${1} - c.y)!))) * (v ** c.y) * ((1 - v) ** ( ${1} - c.y))) AS h
    FROM   controlp AS c, LATERAL (SELECT {2}.x / {0}, {2}.y / {1}) AS _(u,v)$[gridx, gridy, here];
  JUMP loop0_head ()
}
```

```
BLOCK loop0_head ()
[]
{
  condition%0 <- ${0} > {1}$[i, resolution];
  IF condition%0
  THEN GOTO truthy0 ()
  ELSE GOTO falsey0 ()
}
```

```
BLOCK truthy0 ()
[]
{
  EMIT ${0} = {1}$[angle, max_angle];
  STOP
}
```

```
BLOCK falsey0 ()
[]
{
  loc <- $({0}.x + {1}.x, {0}.y + {1}.y) :: point$[loc, step];
  GOTO inter8 ()
}
```

```
BLOCK inter8 ()
[]
{
  hloc <- $SELECT SUM(( ${0}! / ((c.x!) * (( ${0} - c.x)!))) * (u ** c.x) * ((1 - u) ** ( ${0} - c.x)) *
    ( ${1}! / ((c.y!) * (( ${1} - c.y)!))) * (v ** c.y) * ((1 - v) ** ( ${1} - c.y))) AS h
    FROM   controlp AS c, LATERAL (SELECT {2}.x / {0}, {2}.y / {1}) AS _(u,v)$[gridx, gridy, loc];
  GOTO inter9 ()
}
```

```
BLOCK inter9 ()
[]
{
  angle <- $degrees(atan(( ${0} - {1}) / sqrt(( ${2}.x - {1}.x)** 2 + ( ${2}.y - {1}.y)** 2)))$[hloc, hhere, loc];
  GOTO inter10 ()
}
```

```
BLOCK inter10 ()
[]
{
  condition%1 <- ${0} IS NULL OR {1} > {2}$[max_angle, angle, max_angle];
  IF condition%1
  THEN GOTO truthy1 ()
  ELSE JUMP loop0_head ()
}
```

```
BLOCK truthy1 ()
[]
{
  max_angle <- ${0}$[angle];
  JUMP loop0_head ()
}
```



```
here <- $(x,0)$[]
there <- $(x,10)$[]
gridx <- $10$[]
gridy <- $10$[]
resolution <- $16$[]
```

```
BLOCK entry (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[]
{
  gridx <- ${0} - 1$[gridx];
  gridy <- ${0} - 1$[gridy];
  step <- $(( ${0}.x - {1}.x) / {2}, ({0}.y - {1}.y) / {2}) :: point$[there, here, resolution];
  loc <- ${0}$[here];
  max_angle <- $NULL :: float$[];
  i <- $1$[];
  GOTO inter0 (angle, loc, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK inter0 (angle, loc, there, resolution, gridx, max_angle, step, gridy, here, i)
[]
{
  hhere <- $SELECT SUM(({0}! / ((c.x!) * (({0} - c.x)!))) * (u ** c.x) * ((1 - u) ** ({0} - c.x)) *
    ({1}! / ((c.y!) * (({1} - c.y)!))) * (v ** c.y) * ((1 - v) ** ({1} - c.y))) AS h
    FROM controlp AS c, LATERAL (SELECT {2}.x / {0}, {2}.y / {1}) AS _(u,v)$[gridx, gridy, here];
  JUMP loop0_head (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK loop0_head (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[]
{
  condition%0 <- ${0} > {1}$[i, resolution];
  IF condition%0
  THEN GOTO truthy0 (angle, max_angle)
  ELSE GOTO falsey0 (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK truthy0 (angle, max_angle)
[]
{
  EMIT ${0} = {1}$[angle, max_angle];
  STOP
}
```

```
BLOCK falsey0 (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[]
{
  loc <- $({0}.x + {1}.x, {0}.y + {1}.y) :: point$[loc, step];
  GOTO inter8 (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK inter8 (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[]
{
  hloc <- $SELECT SUM(({0}! / ((c.x!) * (({0} - c.x)!))) * (u ** c.x) * ((1 - u) ** ({0} - c.x)) *
    ({1}! / ((c.y!) * (({1} - c.y)!))) * (v ** c.y) * ((1 - v) ** ({1} - c.y))) AS h
    FROM controlp AS c, LATERAL (SELECT {2}.x / {0}, {2}.y / {1}) AS _(u,v)$[gridx, gridy, loc];
  GOTO inter9 (hhere, loc, there, resolution, gridx, max_angle, step, gridy, here, i, hloc)
}
```

```
BLOCK inter9 (hhere, loc, there, resolution, gridx, max_angle, step, gridy, here, i, hloc)
[]
{
  angle <- $degrees(atan(({0} - {1}) / sqrt(({2}.x - {1}.x)** 2 + ({2}.y - {1}.y)** 2))$[hloc, hhere, loc];
  GOTO inter10 (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK inter10 (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[]
{
  condition%1 <- ${0} IS NULL OR {1} > {2}$[max_angle, angle, max_angle];
  IF condition%1
  THEN GOTO truthy1 (angle, loc, hhere, there, resolution, gridx, step, gridy, here, i)
  ELSE JUMP loop0_head (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK truthy1 (angle, loc, hhere, there, resolution, gridx, step, gridy, here, i)
[]
{
  max_angle <- ${0}$[angle];
  JUMP loop0_head (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
here <- $(x,0)$[]
there <- $(x,10)$[]
gridx <- $10$[]
gridy <- $10$[]
resolution <- $16$[]
angle <- $NULL$[]
loc <- $NULL$[]
hhere <- $NULL$[]
max_angle <- $NULL$[]
step <- $NULL$[]
i <- $NULL$[]
```

```
BLOCK entry (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[]
{
  gridx <- ${0} - 1$[gridx];
  gridy <- ${0} - 1$[gridy];
  step <- $(( ${0}.x - {1}.x) / {2}, ( ${0}.y - {1}.y) / {2}) :: point$[there, here, resolution];
  loc <- ${0}$[here];
  max_angle <- $NULL :: float$[];
  i <- $1$[];
  GOTO inter0 (angle, loc, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK inter0 (angle, loc, there, resolution, gridx, max_angle, step, gridy, here, i)
[]
{
  hhere <- $SELECT SUM(( ${0}! / ((c.x!) * (( ${0} - c.x)!))) * (u ** c.x) * ((1 - u) ** ( ${0} - c.x)) *
    ( ${1}! / ((c.y!) * (( ${1} - c.y)!))) * (v ** c.y) * ((1 - v) ** ( ${1} - c.y))) AS h
    FROM   controlp AS c, LATERAL (SELECT {2}.x / {0}, {2}.y / {1}) AS _(u,v)$[gridx, gridy, here];
  JUMP loop0_head (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK loop0_head (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[]
{
  condition%0 <- ${0} > {1}$[i, resolution];
  IF condition%0
  THEN GOTO truthy0 (angle, max_angle)
  ELSE GOTO falsey0 (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK truthy0 (angle, max_angle)
[]
{
  EMIT ${0} = {1}$[angle, max_angle];
  STOP
}
```

```
BLOCK falsey0 (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[]
{
  loc <- $({0}.x + {1}.x, {0}.y + {1}.y) :: point$[loc, step];
  GOTO inter8 (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK inter8 (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[]
{
  hloc <- $SELECT SUM(( ${0}! / ((c.x!) * (( ${0} - c.x)!))) * (u ** c.x) * ((1 - u) ** ( ${0} - c.x)) *
    ( ${1}! / ((c.y!) * (( ${1} - c.y)!))) * (v ** c.y) * ((1 - v) ** ( ${1} - c.y))) AS h
    FROM   controlp AS c, LATERAL (SELECT {2}.x / {0}, {2}.y / {1}) AS _(u,v)$[gridx, gridy, loc];
  GOTO inter9 (hhere, loc, there, resolution, gridx, max_angle, step, gridy, here, i, hloc)
}
```

```
BLOCK inter9 (hhere, loc, there, resolution, gridx, max_angle, step, gridy, here, i, hloc)
[]
{
  angle <- $degrees(atan(( ${0} - {1}) / sqrt(({2}.x - {1}.x)** 2 + ({2}.y - {1}.y)** 2))$[hloc, hhere, loc];
  GOTO inter10 (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK inter10 (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[]
{
  condition%1 <- ${0} IS NULL OR {1} > {2}$[max_angle, angle, max_angle];
  IF condition%1
  THEN GOTO truthy1 (angle, loc, hhere, there, resolution, gridx, step, gridy, here, i)
  ELSE JUMP loop0_head (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK truthy1 (angle, loc, hhere, there, resolution, gridx, step, gridy, here, i)
[]
{
  max_angle <- ${0}$[angle];
  JUMP loop0_head (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
here <- $(x,0)$[]
there <- $(x,10)$[]
gridx <- $10$[]
gridy <- $10$[]
resolution <- $16$[]
angle <- $NULL$[]
loc <- $NULL$[]
hhere <- $NULL$[]
max_angle <- $NULL$[]
step <- $NULL$[]
i <- $NULL$[]
```

```
BLOCK entry (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[
  FROM LOOP AS entry
]
{
  gridx <- ${0} - 1$[gridx];
  gridy <- ${0} - 1$[gridy];
  step <- $(( ${0}.x - ${1}.x) / {2}, ( ${0}.y - ${1}.y) / {2}) :: point$[there, here, resolution];
  loc <- ${0}$[here];
  max_angle <- $NULL :: float$[];
  i <- $1$[];
  GOTO inter0 (angle, loc, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK inter0 (angle, loc, there, resolution, gridx, max_angle, step, gridy, here, i)
[
  FROM entry
  WITH (angle, loc, there, resolution, gridx, max_angle, step, gridy, here, i)
  WHERE TRUE
]
{
  hhere <- $SELECT SUM(( ${0}! / ((c.x!) * (( ${0} - c.x)!))) * (u ** c.x) * ((1 - u) ** ( ${0} - c.x)) *
    ( ${1}! / ((c.y!) * (( ${1} - c.y)!))) * (v ** c.y) * ((1 - v) ** ( ${1} - c.y))) AS h
    FROM controlp AS c, LATERAL (SELECT {2}.x / {0}, {2}.y / {1}) AS _(u,v)$[gridx, gridy, here];
  JUMP loop0_head (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK loop0_head (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[
  FROM LOOP AS loop0_head
]
{
  condition%0 <- ${0} > {1}$[i, resolution];
  IF condition%0
  THEN GOTO truthy0 (angle, max_angle)
  ELSE GOTO falsey0 (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK truthy0 (angle, max_angle)
[
  FROM loop0_head
  WITH (angle, max_angle)
  WHERE (TRUE) AND (condition%0)
]
{
  EMIT ${0} = {1}$[angle, max_angle];
  STOP
}
```

```
BLOCK falsey0 (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[
  FROM loop0_head
  WITH (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
  WHERE (TRUE) AND (NOT (condition%0))
]
{
  loc <- $({0}.x + {1}.x, {0}.y + {1}.y) :: point$[loc, step];
  GOTO inter8 (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK inter8 (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[
  FROM falsey0
  WITH (loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
  WHERE TRUE
]
{
  hloc <- $SELECT SUM(( ${0}! / ((c.x!) * (( ${0} - c.x)!))) * (u ** c.x) * ((1 - u) ** ( ${0} - c.x)) *
    ( ${1}! / ((c.y!) * (( ${1} - c.y)!))) * (v ** c.y) * ((1 - v) ** ( ${1} - c.y))) AS h
    FROM controlp AS c, LATERAL (SELECT {2}.x / {0}, {2}.y / {1}) AS _(u,v)$[gridx, gridy, loc];
  GOTO inter9 (hhere, loc, there, resolution, gridx, max_angle, step, gridy, here, i, hloc)
}
```

```
BLOCK inter9 (hhere, loc, there, resolution, gridx, max_angle, step, gridy, here, i, hloc)
[
  FROM inter8
  WITH (hhere, loc, there, resolution, gridx, max_angle, step, gridy, here, i, hloc)
  WHERE TRUE
]
{
  angle <- $degrees(atan(( ${0} - ${1}) / sqrt(({2}.x - ${1}.x)** 2 + ({2}.y - ${1}.y)** 2)))$[hloc, hhere, loc];
  GOTO inter10 (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK inter10 (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
[
  FROM inter9
  WITH (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
  WHERE TRUE
]
{
  condition%1 <- ${0} IS NULL OR {1} > {2}$[max_angle, angle, max_angle];
  IF condition%1
  THEN GOTO truthy1 (angle, loc, hhere, there, resolution, gridx, step, gridy, here, i)
  ELSE JUMP loop0_head (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```

```
BLOCK truthy1 (angle, loc, hhere, there, resolution, gridx, step, gridy, here, i)
[
  FROM inter10
  WITH (angle, loc, hhere, there, resolution, gridx, step, gridy, here, i)
  WHERE (TRUE) AND (condition%1)
]
{
  max_angle <- ${0}$[angle];
  JUMP loop0_head (angle, loc, hhere, there, resolution, gridx, max_angle, step, gridy, here, i)
}
```